

Object-Oriented Programming in Java

Comprehensive Lecture Notes: Classes, Objects & Java Fundamentals

Course: Object-Oriented Programming

Topic: OOP Fundamentals, Static Features, Variables, Predefined Classes & Packages

Date: [Insert Date]

☐ Table of Contents

1. [Learning Objectives](#)
 2. [Part 1: OOP Fundamentals Review \(Circle Example\)](#)
 3. [Part 2: Static Methods & Variables](#)
 4. [Part 3: Variable Types & Scopes](#)
 5. [Part 4: Java Predefined Classes](#)
 6. [Part 5: Packages & Imports](#)
 7. [Summary & Quick Reference](#)
 8. [Practice Exercises](#)
 9. [Common Mistakes to Avoid](#)
-

Learning Objectives

By the end of this lecture, you should be able to:

- ✓ Define and create classes with proper constructors
 - ✓ Explain object assignment and reference semantics
 - ✓ Implement method overloading correctly
 - ✓ Distinguish between instance, static, and local variables
 - ✓ Use static methods and understand static context rules
 - ✓ Work effectively with String, StringBuilder, and Math classes
 - ✓ Organize code using packages and imports
-

Part 1: OOP Fundamentals Review

Using Circle as an Example

1.1 Complete Circle Class

Here's a complete Circle class demonstrating all fundamental OOP concepts:

```
package com.iit.shapes;

/**
 * Represents a circle with radius.
 * Demonstrates: classes, objects, constructors, encapsulation,
 * method overloading, and the 'this' keyword.
 */
public class Circle {

    // =====
    // FIELDS (Instance and Class Variables)
    // =====

    /**
     * Instance variable - each Circle object has its own radius value
     * Private = encapsulation (can only be accessed within this class)
     */
    private double radius;

    /**
     * Static constant - shared by ALL Circle objects
     * public = accessible everywhere
     * static = belongs to the class, not individual objects
     * final = cannot be changed (constant)
     */
    public static final double PI = Math.PI;

    // =====
    // CONSTRUCTORS - Initialize object state
    // =====

    /**
     * No-argument constructor - creates a unit circle (radius = 1.0)
     * Constructor chaining: calls the other constructor using this()
     */
    public Circle() {
        this(1.0); // Must be first statement
    }

    /**
     * Parameterized constructor - creates circle with specified radius
     * @param radius The radius value (must be non-negative)
     */
    public Circle(double radius) {
        setRadius(radius); // Use setter for validation
    }

    // =====
    // ACCESSOR & MUTATOR METHODS (Getters & Setters)
    // =====

    /**
     * Getter - returns the radius value
     * @return current radius
     */
    public double getRadius() {
        return radius;
    }
}
```

```

/**
 * Setter - sets the radius with validation
 * @param radius new radius value
 * @throws IllegalArgumentException if radius is negative
 */
public void setRadius(double radius) {
    if (radius < 0) {
        throw new IllegalArgumentException("Radius must be >= 0, got: "
+ radius);
    }
    this.radius = radius; // 'this.radius' = field, 'radius' =
parameter
}

// =====
// INSTANCE METHODS - Operate on specific object's data
// =====

/**
 * Calculates area of this circle
 * @return area ( $\pi \times r^2$ )
 */
public double getArea() {
    return PI * radius * radius;
}

/**
 * Calculates circumference of this circle
 * @return circumference ( $2 \times \pi \times r$ )
 */
public double getCircumference() {
    return 2 * PI * radius;
}

// =====
// METHOD OVERLOADING - Same name, different parameters
// =====

/**
 * Scales the circle by a decimal factor
 * @param factor Multiplier (e.g., 2.0 doubles the radius)
 */
public void scale(double factor) {
    if (factor <= 0) {
        throw new IllegalArgumentException("Scale factor must be
positive");
    }
    this.radius *= factor;
}

/**
 * Scales the circle by a percentage
 * @param percent Percentage as integer (e.g., 150 means multiply by
1.5)
 */
public void scale(int percent) {
    if (percent <= 0) {
        throw new IllegalArgumentException("Percent must be positive");
    }
    // CRITICAL: Use 100.0 (not 100) to force double division!
    // 150 / 100 = 1 (integer division - WRONG!)

```

```

        // 150 / 100.0 = 1.5 (double division - CORRECT!)
        this.radius *= (percent / 100.0);
    }

    // =====
    // UTILITY METHODS
    // =====

    /**
     * Returns string representation of this circle
     * @return formatted string with radius and area
     */
    @Override
    public String toString() {
        return String.format("Circle[radius=%.2f, area=%.2f]", radius,
getArea());
    }
}

```

1.2 Key Concepts Explained

A. Class vs Object

```

// Circle class = BLUEPRINT (template/recipe)
// It defines what a circle IS and what it can DO

Circle c1 = new Circle(5.0); // c1 = OBJECT (actual circle instance)
Circle c2 = new Circle(3.0); // c2 = another OBJECT (different circle)

```

Analogy:

- **Class** = Cookie cutter (the template)
- **Object** = Actual cookie (created from the template)

Memory Diagram:

Class Circle (Blueprint)

```

- radius: double
+ PI: double
+ Circle()
+ getArea()
+ scale()

```

Heap Memory (Objects)

```

Circle object #1
radius = 5.0

```

```

Circle object #2
radius = 3.0

```

B. Constructors

Purpose: Initialize object state when created

Key Rules:

1. Same name as the class

2. No return type (not even `void`)
3. Called automatically with `new`
4. Can be overloaded (multiple constructors)

Constructor Chaining:

```
public Circle() {  
    this(1.0); // Calls Circle(double radius)  
}
```

- `this()` must be the **first statement**
 - Reduces code duplication
 - Centralizes initialization logic
-

C. The `this` Keyword

The `this` keyword refers to the **current object** (the one the method was called on).

Three Uses of `this`:

```
public class Circle {  
    private double radius;  
  
    // USE 1: Distinguish field from parameter  
    public void setRadius(double radius) {  
        this.radius = radius; // this.radius = field, radius = parameter  
    }  
  
    // USE 2: Call another instance method  
    public void printInfo() {  
        System.out.println(this.getArea()); // Can omit 'this.' here  
    }  
  
    // USE 3: Call another constructor  
    public Circle() {  
        this(1.0); // Calls Circle(double)  
    }  
}
```

When you call: `circle1.setRadius(7.5)`

Inside the method: `this = circle1`

D. Object Assignment and References

❏ CRITICAL CONCEPT: Variables store REFERENCES (memory addresses), not objects!

```
Circle c1 = new Circle(5.0);  
Circle c2 = c1; // ⚠ COPIES THE REFERENCE, NOT THE OBJECT!
```

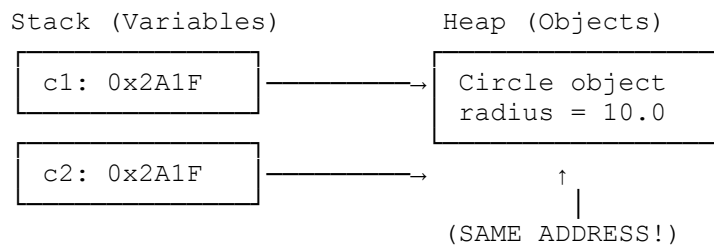
```
c2.setRadius(10.0); // Modifies the object

System.out.println(c1.getRadius()); // Prints 10.0 (!)
System.out.println(c2.getRadius()); // Prints 10.0
```

Why does c1 show 10.0?

Because c1 and c2 point to the **same object** in memory!

Memory Diagram:



This is called ALIASING - two variables referring to the same object.

Contrast with Primitives:

```
int x = 5;
int y = x; // COPIES THE VALUE
y = 10;
// x is still 5 (separate values)

Circle c1 = new Circle(5);
Circle c2 = c1; // COPIES THE REFERENCE
c2.setRadius(10);
// c1.getRadius() returns 10 (same object)
```

E. Method Overloading

Definition: Multiple methods with the **same name** but **different parameter lists**.

```
public void scale(double factor) { // Version 1: takes double
    this.radius *= factor;
}

public void scale(int percent) { // Version 2: takes int
    this.radius *= (percent / 100.0);
}
```

How the compiler chooses:

```
Circle c = new Circle(10);
c.scale(2.0); // Calls scale(double) because 2.0 is a double
c.scale(150); // Calls scale(int) because 150 is an int
```

What makes methods overloaded:

- ✓ Different number of parameters

- ✓ Different types of parameters
- ✓ Different order of parameter types
- ✗ NOT different return types alone
- ✗ NOT different parameter names alone

Examples:

```
// Valid overloading:
public void draw(int x, int y) { }
public void draw(double x, double y) { }
public void draw(int x, int y, int z) { }

// NOT valid overloading (compile error):
public void draw(int x, int y) { }
public int draw(int x, int y) { } // ✗ Only return type differs
```

⚠ Common Bug in scale(int percent):

```
// WRONG:
this.radius *= (percent / 100); // 150 / 100 = 1 (integer division!)

// CORRECT:
this.radius *= (percent / 100.0); // 150 / 100.0 = 1.5 (double division)
```

1.3 Using the Circle Class

Demo Code:

```
import com.iit.shapes.Circle;

public class CircleDemo {
    public static void main(String[] args) {
        // Creating objects
        Circle c1 = new Circle(); // Uses no-arg constructor, radius
= 1.0
        Circle c2 = new Circle(5.0); // Uses parameterized constructor

        // Using instance methods
        System.out.println("c2 area: " + c2.getArea());
        System.out.println("c2 circumference: " + c2.getCircumference());

        // Demonstrating aliasing
        Circle c3 = c2; // Both refer to same object
        c3.setRadius(10.0);
        System.out.println("c2 radius: " + c2.getRadius()); // Prints
10.0!

        // Method overloading
        Circle c4 = new Circle(4.0);
        c4.scale(2.0); // Calls scale(double) → radius becomes 8.0
        c4.scale(50); // Calls scale(int) → radius becomes 4.0 (50% of
8)

        System.out.println(c4); // Uses toString()
    }
}
```

Output:

```
c2 area: 78.53981633974483
c2 circumference: 31.41592653589793
c2 radius: 10.0
Circle[radius=4.00, area=50.27]
```

💡 Key Takeaways: Part 1

Concept	Key Point
Class	Blueprint/template that defines structure and behavior
Object	Actual instance created from the class
Constructor	Special method to initialize object state
this	Reference to the current object
Assignment	Copies references, not objects (aliasing)
Overloading	Same method name, different parameters

Part 2: Static Methods & Variables

Using Cylinder as an Example

2.1 Understanding Static

Static means "belonging to the CLASS itself, not to individual objects."

Comparison:

<u>Instance (Non-Static)</u>	<u>Static</u>
• Belongs to each object • Each object has own copy • Accessed via object reference • Can access both static and instance members	• Belongs to the class • One copy shared by all • Accessed via class name • Can ONLY access static members

2.2 Complete Cylinder Class

```
package com.iit.shapes;

/**
 * Represents a cylinder with radius and height.
 * Demonstrates static variables, static methods, and the distinction
 * between instance and class members.
 */
public class Cylinder {

    // =====
    // INSTANCE VARIABLES (Per-Object State)
```



```

// =====
private double radius;    // Each cylinder has its own radius
private double height;    // Each cylinder has its own height

// =====
// STATIC VARIABLES (Class-Level State - Shared by All)
// =====

/**
 * Constant PI - shared by all cylinders
 * public = accessible everywhere
 * static = one copy for the class
 * final = cannot be modified
 */
public static final double PI = Math.PI;

/**
 * Counter tracking total number of Cylinder objects created
 * private = only accessible within this class
 * static = shared counter
 */
private static int instanceCount = 0;

// =====
// CONSTRUCTOR
// =====

public Cylinder(double radius, double height) {
    if (radius <= 0 || height <= 0) {
        throw new IllegalArgumentException("Dimensions must be
positive");
    }
    this.radius = radius;
    this.height = height;

    // Increment the shared counter
    instanceCount++; // Every time we create a Cylinder, count goes up
}

// =====
// INSTANCE METHODS (Operate on Specific Object's Data)
// =====

/**
 * Calculates volume of THIS cylinder
 * Instance method can access:
 * ✓ Instance variables (radius, height)
 * ✓ Static variables (PI, instanceCount)
 */
public double getVolume() {
    return PI * radius * radius * height;
}

/**
 * Calculates lateral surface area (curved surface)
 */
public double getLateralSurfaceArea() {
    return 2 * PI * radius * height;
}

/**

```

```

    * Calculates total surface area (lateral + two circles)
    */
    public double getTotalSurfaceArea() {
        return 2 * PI * radius * (radius + height);
    }

    // =====
    // STATIC METHODS (Belong to Class, Not Objects)
    // =====

    /**
     * Returns total number of Cylinder objects created
     * Static method can access:
     * ✓ Static variables (instanceCount)
     * ✗ CANNOT access instance variables (radius, height)
     * ✗ CANNOT use 'this' keyword
     */
    public static int getInstanceCount() {
        return instanceCount;

        // These would cause COMPILE ERRORS:
        // return radius;          // ✗ non-static variable cannot be
referenced
        // return this.height;    // ✗ cannot use 'this' in static context
    }

    /**
     * Utility method: Calculate volume without creating an object
     * Useful for quick calculations
     */
    public static double calculateVolume(double r, double h) {
        return PI * r * r * h;
    }

    /**
     * Factory method: Creates cylinder from diameter instead of radius
     * Static methods can instantiate objects of their own class
     */
    public static Cylinder fromDiameter(double diameter, double height) {
        return new Cylinder(diameter / 2.0, height);
    }

    // =====
    // ACCESSOR METHODS
    // =====

    public double getRadius() { return radius; }
    public double getHeight() { return height; }

    @Override
    public String toString() {
        return String.format("Cylinder[r=%.2f, h=%.2f, vol=%.2f]",
                               radius, height, getVolume());
    }
}

```

2.3 Static Variables in Detail

Memory Model:

Instance Variables (Each object has own copy):

Cylinder #1 radius: 2.0 height: 5.0	Cylinder #2 radius: 3.0 height: 10.0	Cylinder #3 radius: 5.0 height: 8.0
---	--	---

Static Variables (One copy shared by ALL):

Cylinder class data (Method Area) • PI = 3.14159... • instanceCount = 3

↑
Cylinder #1

↑
Cylinder #2

↑
Cylinder #3

When to use static variables:

1. **Constants** that don't change: PI, MAX_SIZE
2. **Counters** tracking instances: instanceCount
3. **Shared configuration**: defaultCapacity
4. **Caches** shared by all instances

⚠ **Warning:** Avoid mutable static state in most cases (can cause bugs, testing issues).

2.4 Static Methods in Detail

The Golden Rule:

Static methods CANNOT access instance members
Why? They don't belong to any specific object!

What static methods CAN and CANNOT do:

```
public class Cylinder {  
    private double radius;           // instance variable  
    private static int count = 0;    // static variable  
  
    public double getVolume() {      // instance method  
        return PI * radius * height; // ✓ can access instance variables  
        return count;                // ✓ can access static variables  
    }  
  
    public static void staticMethod() { // static method  
        // return radius;             // ✗ ERROR: non-static variable  
        // this.radius = 5;           // ✗ ERROR: cannot use 'this'  
        return count;                // ✓ can access static variables  
        staticMethod2();              // ✓ can call static methods  
    }  
}
```

```

        // getVolume();
method
    }
}
// ✗ ERROR: cannot call instance

```

Access Rules Matrix:

	Instance Method	Static Method
Instance variable	✓	✗
Static variable	✓	✓
Instance method	✓	✗
Static method	✓	✓
'this' keyword	✓	✗

2.5 Using Static Members

Demo Code:

```

import com.iit.shapes.Cylinder;

public class CylinderDemo {
    public static void main(String[] args) {
        // Creating Cylinder objects
        Cylinder cyl = new Cylinder(2.0, 5.0);
        Cylinder cy2 = new Cylinder(3.0, 10.0);
        Cylinder cy3 = new Cylinder(1.5, 7.0);

        // =====
        // INSTANCE METHODS - Need an object to call
        // =====
        System.out.println("cyl volume: " + cyl.getVolume());
        System.out.println("cy2 surface: " + cy2.getTotalSurfaceArea());

        // =====
        // STATIC ACCESS - Via class name (preferred style)
        // =====

        // ✓ CORRECT: Access static members via class name
        System.out.println("Total cylinders: " +
Cylinder.getInstanceCount());
        System.out.println("PI value: " + Cylinder.PI);

        // ⚠ WORKS BUT BAD STYLE: Accessing static via instance
        System.out.println("Count via cyl: " + cyl.getInstanceCount());
        // ^ Compiler warning: "static member accessed via instance
reference"
        // Confusing because it LOOKS like it's about cyl, but it's not!

        // =====
        // STATIC UTILITY METHOD - No object needed
        // =====
        double volume = Cylinder.calculateVolume(4.0, 12.0);
        System.out.println("Calculated volume: " + volume);
    }
}

```

```

// =====
// STATIC FACTORY METHOD
// =====
Cylinder cy4 = Cylinder.fromDiameter(10.0, 15.0); // diameter=10 →
radius=5
System.out.println(cy4);
}
}

```

Output:

```

cy1 volume: 62.83185307179586
cy2 surface: 245.04422698228633
Total cylinders: 3
PI value: 3.141592653589793
Count via cy1: 3
Calculated volume: 603.1857894892403
Cylinder[r=5.00, h=15.00, vol=1178.10]

```

2.6 When to Use Static

✓ Good Uses of Static:

```

// 1. CONSTANTS - Values that never change
public static final double EARTH_GRAVITY = 9.81;
public static final int MAX_CAPACITY = 100;

// 2. UTILITY METHODS - Pure functions (no state needed)
public static int max(int a, int b) {
    return (a > b) ? a : b;
}

public static double calculateDistance(double x1, double y1, double x2,
double y2) {
    return Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
}

// 3. FACTORY METHODS - Alternative constructors
public static Cylinder fromDiameter(double d, double h) {
    return new Cylinder(d / 2, h);
}

public static LocalDate parse(String dateStr) {
    // Parse and return date
}

// 4. COUNTERS - Track class-level statistics
private static int totalInstancesCreated = 0;

```

✗ Poor Uses of Static:

```

// BAD: Mutable shared state (hard to test, not thread-safe)
public static List<User> allUsers = new ArrayList<>();
public static int globalCounter = 0;

// BAD: Everything static (defeats OOP purpose)

```

```

public class BadCalculator {
    public static int value;
    public static void add(int x) { value += x; }
    public static void subtract(int x) { value -= x; }
}
// ^ Should be instance-based!

// BAD: Static in most business objects
public class Customer {
    private static String name; // ✗ Each customer should have own name!
}

```

Rule of Thumb:

If it doesn't need object state → consider static

If multiple instances need different values → use instance

💡 Key Takeaways: Part 2

Concept	Key Point
Static Variable	One copy shared by all instances of the class
Static Method	Belongs to class; cannot access instance members
Access	Use <code>ClassName.member</code> for static, <code>objectName.member</code> for instance
Memory	Instance = heap, Static = method area
When to Use	Constants, utilities, factories, counters

Part 3: Variable Types & Scopes

3.1 Three Types of Variables

Java has three types of variables, each with different characteristics:

```

public class VariableTypes {

    // =====
    // 1. INSTANCE VARIABLES (Fields)
    // =====
    private int instanceVar = 42;           // Belongs to each object
    private String name;                   // Default value: null
    private double[] scores;               // Default value: null

    // =====
    // 2. STATIC VARIABLES (Class Variables)
    // =====
    private static int classVar = 7;        // Shared by all objects
    public static final String APP_NAME = "MyApp"; // Constant

    // =====
    // 3. LOCAL VARIABLES (Method/Block Variables)
    // =====
}

```

```

public int compute(int parameter) { // 'parameter' is a local variable
    int localVar = parameter + instanceVar; // 'localVar' is local

    if (localVar > classVar) {
        // 'blockLocal' only exists inside this if-block
        int blockLocal = localVar - classVar;
        System.out.println(blockLocal); // ✓ OK here
        return blockLocal;
    }

    // System.out.println(blockLocal); // ✗ ERROR: out of scope!

    return localVar;
}

public void demonstrateScope() {
    int x = 10; // Local variable

    { // New block scope
        int y = 20; // Only exists in this block
        System.out.println(x); // ✓ Can access outer scope
        System.out.println(y); // ✓ OK here
    }

    System.out.println(x); // ✓ Still in scope
    // System.out.println(y); // ✗ ERROR: y out of scope
}

public void initializationExample() {
    // Instance and static variables have default values
    System.out.println(instanceVar); // Prints: 42
    System.out.println(name); // Prints: null

    // Local variables MUST be initialized before use
    int local;
    // System.out.println(local); // ✗ ERROR: might not be
initialized

    local = 5; // Initialize
    System.out.println(local); // ✓ OK now
}

```

3.2 Comprehensive Comparison Table

	INSTANCE	STATIC	LOCAL
	VARIABLE	VARIABLE	VARIABLE
Declared method, Where? or	Inside class (outside methods)	Inside class with 'static'	Inside constructor,

			block
Lifetime	While object exists	While program is running	While method/block
executes			
Scope declaration block	Throughout all instance methods	Throughout class (all methods)	From to end of
Default	YES	YES	NO
Value? initialize	(0, false, null)	(0, false, null)	(must before use)
Accessed Via	objectName.var (or this.var)	ClassName.var	Direct name (no prefix)
Memory Location (frame)	Heap (with object)	Method Area (class metadata)	Stack (method
Modifiers Allowed (allowed)	public, private, protected, static, final	public, private, protected, final	None (only final
Example	private int age; private String name;	static int count; static final double PI;	int sum = 0; String name = "John";
Use Case	Object state (properties)	Shared data, constants, utilities	Temporary calculations and storage

3.3 Default Values

Instance and Static Variables:

```
public class Defaults {
    // Numeric types default to 0
    private int number;           // 0
    private double decimal;       // 0.0
    private long bigNumber;       // 0L

    // Boolean defaults to false
    private boolean flag;         // false

    // Object references default to null
    private String text;          // null
    private int[] array;          // null

    public void test() {
        System.out.println(number);    // 0
        System.out.println(decimal);   // 0.0
        System.out.println(flag);      // false
        System.out.println(text);      // null
    }
}
```

Local Variables - NO Default Values:

```
public void method() {
    int x;
    // System.out.println(x); // ✗ COMPILE ERROR: variable x might not
    // have been initialized

    x = 5; // Must initialize
    System.out.println(x); // ✔ OK now
}
```

3.4 Detailed Examples

Example 1: Instance vs Static

```
public class Counter {
    private int instanceCount = 0;    // Each object has own copy
    private static int staticCount = 0; // Shared by all objects

    public void increment() {
        instanceCount++;
        staticCount++;
    }

    public void display() {
        System.out.println("Instance: " + instanceCount);
        System.out.println("Static: " + staticCount);
    }
}
```

```

    }

    public static void main(String[] args) {
        Counter c1 = new Counter();
        Counter c2 = new Counter();

        c1.increment(); // c1: instance=1, static=1
        c1.increment(); // c1: instance=2, static=2

        c2.increment(); // c2: instance=1 (separate!), static=3 (shared!)

        c1.display();
        // Output:
        // Instance: 2
        // Static: 3

        c2.display();
        // Output:
        // Instance: 1
        // Static: 3 (same static value!)
    }
}

```

Example 2: Local Variable Scope

```

public class Scope {
    public void method() {
        int x = 10; // Scope: entire method

        if (x > 5) {
            int y = 20; // Scope: only inside if-block
            System.out.println(x); // ✓ Can access x
            System.out.println(y); // ✓ Can access y
        }

        System.out.println(x); // ✓ Can still access x
        // System.out.println(y); // ✗ ERROR: y out of scope

        for (int i = 0; i < 5; i++) { // i scope: only in loop
            System.out.println(i);
        }

        // System.out.println(i); // ✗ ERROR: i out of scope
    }
}

```

💡 Key Takeaways: Part 3

Variable Type	Lifetime	Scope	Default Value	Memory
Instance	Object exists	Instance methods	YES (0/null/false)	Heap
Static	Program runs	All methods	YES (0/null/false)	Method Area
Local	Method executes	Declaration to block end	NO (must initialize)	Stack

Memory Analogy:

- **Instance variables** = Your personal locker (each student has their own)

- **Static variables** = School bulletin board (one shared by everyone)
 - **Local variables** = Notepad during a test (temporary, discarded after)
-

Part 4: Java Predefined Classes

4.1 String Class

Why String is Special

String is a class (not a primitive), but it's so commonly used that Java gives it special treatment:

- String literals: "hello" (doesn't need new)
 - String pool: Reuses identical literals
 - Immutable: Cannot be changed after creation
-

A. String Immutability

Critical Concept: Strings are **IMMUTABLE** - they NEVER change!

```
public class StringImmutability {
    public static void main(String[] args) {
        // =====
        // STRING METHODS RETURN NEW STRINGS
        // =====

        String s1 = "IIT";
        String s2 = s1.toLowerCase(); // Creates NEW string

        System.out.println(s1); // "IIT" (UNCHANGED!)
        System.out.println(s2); // "iit" (new object)

        // ✖ COMMON MISTAKE:
        String message = "hello";
        message.toUpperCase(); // Does NOTHING - result not
stored! System.out.println(message); // Still "hello"

        // ✔ CORRECT:
        message = message.toUpperCase(); // Must reassign to variable
        System.out.println(message); // "HELLO"

        // =====
        // WHY IMMUTABLE?
        // =====
        // 1. Thread-safe (multiple threads can share)
        // 2. String pool optimization (memory efficient)
        // 3. Security (can't modify passwords/URLs after creation)
        // 4. Hash keys (hashCode never changes)
```

} }

Visual:

```
String s = "hello";  
s.toUpperCase();
```

```
Memory:
"hello"   (original)  ← s still points here
"HELLO"   (new)       ← created but not stored anywhere!
```

Correct:

```
s = s.toUpperCase();
```

```
"hello"  (original)  ← no longer referenced
"HELLO"  (new)       ← s now points here
```

B. String Comparison - The == Trap

❑ **CRITICAL: NEVER use == to compare String content!**

```

public class StringComparison {
    public static void main(String[] args) {
        // =====
        // STRING POOL BEHAVIOR
        // =====
        String a = "hello";
        String b = "hello";
        String c = new String("hello");

        // == compares REFERENCES (memory addresses), not content!
        System.out.println(a == b);           // true (string pool - same
object!)
        System.out.println(a == c);           // false (different objects)
        System.out.println(b == c);           // false

        // .equals() compares CONTENT ✓ THIS IS WHAT YOU WANT!
        System.out.println(a.equals(b));      // true
        System.out.println(a.equals(c));      // true
        System.out.println(b.equals(c));      // true

        // =====
        // CASE-INSENSITIVE COMPARISON
        // =====
        String s1 = "Hello";
        String s2 = "HELLO";
        System.out.println(s1.equals(s2));     // false
        System.out.println(s1.equalsIgnoreCase(s2)); // true
    }
}

```

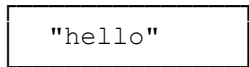
Memory Diagram:

String Pool (for literals):

`"hello"` ← a points here

 ← b also points here (SAME OBJECT!)

Heap (for new String):

 ← c points here (DIFFERENT OBJECT!)

Why does `a == b` return true?

Java optimizes by storing string literals in a "string pool". When you write "hello" multiple times, Java reuses the same object.

Rule: ALWAYS use `.equals()` for String comparison!

C. Common String Methods

```
public class StringMethods {
    public static void main(String[] args) {
        String text = "  Java Programming  ";

        // =====
        // LENGTH & TRIMMING
        // =====
        System.out.println(text.length());           // 21 (includes spaces)
        System.out.println(text.trim());             // "Java Programming"
        (removes leading/trailing)
        System.out.println(text.trim().length());    // 17

        // =====
        // EXTRACTION
        // =====
        System.out.println(text.substring(2, 6));    // "Java" (index 2
to 5)
        System.out.println(text.substring(7));      // "Programming "
        (from index 7 to end)
        System.out.println(text.charAt(2));         // 'J'

        // =====
        // SEARCHING
        // =====
        System.out.println(text.contains("Program")); // true
        System.out.println(text.indexOf("Java"));    // 2
        System.out.println(text.indexOf("Python"));  // -1 (not
found)
        System.out.println(text.startsWith("  Ja")); // true
        System.out.println(text.endsWith("ing  ")); // true

        // =====
        // MODIFICATION (returns NEW strings - original unchanged!)
        // =====
        System.out.println(text.toUpperCase());      // "  JAVA
PROGRAMMING  "
        System.out.println(text.toLowerCase());     // "  java
programming  "
        System.out.println(text.replace("Java", "Python")); // "  Python
Programming  "
```

```

        System.out.println(text.replace(" ", ""));           //
"JavaProgramming"

        // Original is UNCHANGED:
        System.out.println(text); // "  Java Programming  "

        // =====
        // SPLITTING & JOINING
        // =====
        String csv = "one,two,three,four";
        String[] parts = csv.split(","); // Creates array: ["one", "two",
"three", "four"]
        System.out.println(parts[1]);    // "two"
        System.out.println(parts.length); // 4

        // Join array back to string (Java 8+)
        String joined = String.join("-", parts);
        System.out.println(joined); // "one-two-three-four"

        // =====
        // COMPARING
        // =====
        String s1 = "apple";
        String s2 = "banana";
        System.out.println(s1.compareTo(s2)); // Negative (apple < banana
alphabetically)
        System.out.println(s2.compareTo(s1)); // Positive (banana > apple)
        System.out.println(s1.compareTo("apple")); // 0 (equal)
    }
}

```

4.2 StringBuilder Class

The Performance Problem with String

```

// ✗ INEFFICIENT - Creates 1000+ temporary String objects!
public static String badConcat() {
    String result = "";
    for (int i = 0; i < 1000; i++) {
        result += i + " "; // Each += creates a NEW string
    }
    return result;
}

```

Why is this slow?

```

Iteration 0: "" (1 String object)
Iteration 1: "" + "0 " = "0 " (2 String objects)
Iteration 2: "0 " + "1 " = "0 1 " (3 String objects)
Iteration 3: "0 1 " + "2 " = "0 1 2 " (4 String objects)
...
Total after 1000 iterations: ~500,000 String objects created!

```

The Solution: StringBuilder

StringBuilder is **MUTABLE** - it can be modified without creating new objects.

```
// ✓ EFFICIENT - StringBuilder is mutable
public static String goodConcat() {
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < 1000; i++) {
        sb.append(i).append(" "); // Modifies SAME object
    }
    return sb.toString(); // Convert to String at end
}
```

Performance Benchmark

```
public class PerformanceTest {
    public static void main(String[] args) {
        int n = 10000;

        // Test String +=
        long start = System.nanoTime();
        String s = "";
        for (int i = 0; i < n; i++) {
            s += "x";
        }
        long end = System.nanoTime();
        System.out.println("String +=: " + (end - start) / 1_000_000 + "
ms");

        // Test StringBuilder
        start = System.nanoTime();
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            sb.append("x");
        }
        end = System.nanoTime();
        System.out.println("StringBuilder: " + (end - start) / 1_000_000 +
" ms");
    }
}
```

Typical Output:

```
String +=: 189 ms
StringBuilder: 1 ms
```

StringBuilder is ~200x faster!

StringBuilder Methods

```
public class StringBuilderDemo {
    public static void main(String[] args) {
        // =====
        // CREATING STRINGBUILDER
        // =====
    }
}
```

```

        StringBuilder sb1 = new StringBuilder();           // Empty,
capacity 16
        StringBuilder sb2 = new StringBuilder(100);       // Empty,
capacity 100
        StringBuilder sb3 = new StringBuilder("Hello");   // From
String

// =====
// MODIFYING CONTENT
// =====
        StringBuilder sb = new StringBuilder("Hello");

        sb.append(" World");           // "Hello World"
        sb.append("!");                 // "Hello World!"

        // Method chaining (returns 'this')
        sb.append(" Java").append(" is").append(" fun");
        // "Hello World! Java is fun"

        sb.insert(5, ",");              // "Hello, World! Java is fun"
        sb.delete(5, 6);                // "Hello World! Java is fun"
(remove comma)
        sb.replace(0, 5, "Hi");         // "Hi World! Java is fun"
        sb.reverse();                  // "nuf si avaJ !dlroW iH"
        sb.reverse();                  // Back to "Hi World! Java is
fun"

// =====
// QUERIES
// =====
        System.out.println(sb.length());           // Current length
        System.out.println(sb.capacity());         // Internal buffer size
        System.out.println(sb.charAt(0));          // 'H'
        System.out.println(sb.substring(0, 2));    // "Hi"

// =====
// CLEAR CONTENT
// =====
        sb.setLength(0); // Clears all content

// =====
// CONVERT TO STRING
// =====
        sb.append("Final result");
        String result = sb.toString(); // Convert to String
        System.out.println(result);
    }
}

```

StringBuilder vs StringBuffer

```

// StringBuilder - FAST, NOT thread-safe
StringBuilder sb = new StringBuilder();
sb.append("Hello"); // Fast but not synchronized

// StringBuffer - SLOWER, thread-safe (synchronized)
StringBuffer sbuf = new StringBuffer();
sbuf.append("Hello"); // Slower but safe for multiple threads

```


When to use:

- **StringBuilder**: 99% of the time (single-threaded)
- **StringBuffer**: Only when multiple threads access the same buffer

When to Use String vs StringBuilder

```
// ✓ String is fine for simple operations
String greeting = "Hello " + name + "!";
String path = basePath + "/" + filename;

// ✓ Use StringBuilder for loops or many concatenations
StringBuilder html = new StringBuilder();
for (Item item : items) {
    html.append("<li>").append(item.getName()).append("</li>");
}

// ✗ DON'T use StringBuilder unnecessarily
StringBuilder sb = new StringBuilder();
sb.append("Hello");
sb.append(" ");
sb.append("World");
String result = sb.toString();
// ^ Just use: String result = "Hello World";
```

4.3 Math Class

The Math class provides mathematical utilities. All methods are **static** (no need to create Math object).

```
public class MathDemo {
    public static void main(String[] args) {
        // =====
        // CONSTANTS
        // =====
        System.out.println(Math.PI);    // 3.141592653589793
        System.out.println(Math.E);     // 2.718281828459045 (Euler's number)

        // =====
        // BASIC OPERATIONS
        // =====
        System.out.println(Math.abs(-42));    // 42 (absolute value)
        System.out.println(Math.abs(-3.7));   // 3.7

        System.out.println(Math.max(10, 20)); // 20
        System.out.println(Math.min(5, 7));   // 5

        // =====
        // ROUNDING
        // =====
        System.out.println(Math.round(3.7));  // 4 (returns long)
        System.out.println(Math.round(3.4));  // 3
        System.out.println(Math.round(-3.7)); // -4
    }
}
```

```

System.out.println(Math.ceil(3.2));    // 4.0 (rounds up)
System.out.println(Math.ceil(-3.2));   // -3.0

System.out.println(Math.floor(3.9));   // 3.0 (rounds down)
System.out.println(Math.floor(-3.2));  // -4.0

// =====
// POWERS & ROOTS
// =====
System.out.println(Math.pow(2, 3));    // 8.0 (2³)
System.out.println(Math.pow(5, 2));    // 25.0 (5²)
System.out.println(Math.sqrt(16));     // 4.0 (square root)
System.out.println(Math.cbrt(27));     // 3.0 (cube root)

// =====
// RANDOM NUMBERS
// =====
double rand = Math.random();           // [0.0, 1.0) - random decimal

// Random integer from 1 to 6 (dice roll)
int dice = (int)(Math.random() * 6) + 1;

// Random integer from 1 to 100
int random100 = (int)(Math.random() * 100) + 1;

// General formula: [min, max]
// (int)(Math.random() * (max - min + 1)) + min

// =====
// TRIGONOMETRY (angles in RADIANS!)
// =====
System.out.println(Math.sin(Math.PI / 2)); // 1.0 (sin 90°)
System.out.println(Math.cos(0));           // 1.0
System.out.println(Math.tan(Math.PI / 4)); // 1.0 (tan 45°)

// Convert degrees to radians
double radians = Math.toRadians(90);
System.out.println(Math.sin(radians));     // 1.0

// =====
// OTHER USEFUL METHODS
// =====
System.out.println(Math.signum(-5));       // -1.0 (sign of number)
System.out.println(Math.signum(5));        // 1.0
System.out.println(Math.signum(0));        // 0.0
    }
}

```

4.4 Wrapper Classes

What are Wrapper Classes?

Wrapper classes convert primitives to objects (and vice versa).

Primitive Type → Wrapper Class

byte	→ Byte
short	→ Short
int	→ Integer
long	→ Long
float	→ Float
double	→ Double
char	→ Character
boolean	→ Boolean

Why Do We Need Them?

1. **Collections** only store objects (not primitives)
 2. **Utility methods** for conversion and parsing
 3. **Constants** like MAX_VALUE, MIN_VALUE
 4. **Nullable** primitives (null is not possible with primitives)
-

Complete Wrapper Class Examples

```
public class WrapperDemo {
    public static void main(String[] args) {
        // =====
        // PARSING STRINGS TO PRIMITIVES
        // =====
        int age = Integer.parseInt("25");
        double pi = Double.parseDouble("3.14");
        boolean flag = Boolean.parseBoolean("true");
        long big = Long.parseLong("1000000");

        System.out.println(age);    // 25
        System.out.println(pi);     // 3.14
        System.out.println(flag);   // true

        // Handle invalid input
        try {
            int x = Integer.parseInt("abc"); // Throws
NumberFormatException
        } catch (NumberFormatException e) {
            System.out.println("Invalid number format!");
        }

        // =====
        // CONVERTING TO STRINGS
        // =====
        String ageStr = String.valueOf(25);        // "25"
        String piStr = Double.toString(3.14);      // "3.14"
        String flagStr = Boolean.toString(true);    // "true"

        // Alternative (using concatenation)
        String s = "" + 25; // "25" (quick but less clear)

        // =====
        // WRAPPER CLASS CONSTANTS
        // =====
        System.out.println(Integer.MAX_VALUE);    // 2147483647
        System.out.println(Integer.MIN_VALUE);    // -2147483648
    }
}
```

```

System.out.println(Double.MAX_VALUE);    // 1.7976931348623157E308
System.out.println(Double.MIN_VALUE);    // 4.9E-324
System.out.println(Character.MIN_VALUE); // '\u0000'
System.out.println(Character.MAX_VALUE); // '\uffff'

// =====
// AUTOBOXING & UNBOXING (Automatic conversion)
// =====

// Autoboxing: primitive → wrapper (automatic)
Integer obj = 42;        // Compiler: Integer obj =
Integer.valueOf(42);
Double dObj = 3.14;      // Compiler: Double dObj =
Double.valueOf(3.14);

// Unboxing: wrapper → primitive (automatic)
int num = obj;           // Compiler: int num = obj.intValue();
double d = dObj;         // Compiler: double d = dObj.doubleValue();

// Practical use in collections
List<Integer> numbers = new ArrayList<>();
numbers.add(5);          // Autoboxing: int → Integer
numbers.add(10);
int first = numbers.get(0); // Unboxing: Integer → int

// =====
// UTILITY METHODS
// =====

// Integer utilities
System.out.println(Integer.toBinaryString(10)); // "1010"
System.out.println(Integer.toHexString(255));  // "ff"
System.out.println(Integer.toOctalString(8));   // "10"
System.out.println(Integer.compare(5, 10));     // -1 (5 < 10)
System.out.println(Integer.compare(10, 5));     // 1 (10 > 5)
System.out.println(Integer.compare(5, 5));      // 0 (equal)

// Character utilities
System.out.println(Character.isDigit('5'));     // true
System.out.println(Character.isLetter('A'));    // true
System.out.println(Character.isLetterOrDigit('!')); // false
System.out.println(Character.isUpperCase('A')); // true
System.out.println(Character.isLowerCase('a')); // true
System.out.println(Character.toUpperCase('a')); // 'A'
System.out.println(Character.toLowerCase('Z')); // 'z'
System.out.println(Character.isWhitespace(' ')); // true

// Double utilities
System.out.println(Double.isNaN(0.0 / 0.0));     // true (Not a
Number)
System.out.println(Double.isInfinite(1.0 / 0.0)); // true
}
}

```

Autoboxing/Unboxing in Detail

```

// Before Java 5 (manual conversion)
Integer obj = new Integer(42); // Primitive → wrapper
int num = obj.intValue();      // Wrapper → primitive

```

```
// Java 5+ (automatic)
Integer obj = 42;      // Autoboxing
int num = obj;         // Unboxing

// Practical example
List<Integer> list = new ArrayList<>();
list.add(5);           // Autoboxing: int 5 → Integer object
int x = list.get(0);   // Unboxing: Integer object → int 5

// Be careful with null!
Integer obj = null;
// int x = obj; // ✖ NullPointerException at runtime!
```

💡 Key Takeaways: Part 4

Class	Key Points
String	Immutable; use <code>.equals()</code> not <code>==</code> ; methods return new strings
StringBuilder	Mutable; use for loops/many concatenations; convert with <code>.toString()</code>
Math	All static methods; common utilities for calculations
Wrappers	Convert primitives ↔ objects; parsing, constants, autoboxing

Part 5: Packages & Imports

5.1 What are Packages?

Definition: Packages are Java's way of organizing classes into namespaces (like folders organize files).

Purpose:

1. **Avoid name conflicts:** `com.google.Maps` VS `com.apple.Maps`
 2. **Logical organization:** Group related classes together
 3. **Access control:** Package-private visibility
 4. **Modularity:** Easier to maintain and understand code
-

5.2 Package Naming Convention

Reverse domain name: `com.company.project.module`

Examples:

└─ <code>com.iit.shapes</code>	→ <code>Circle</code> , <code>Cylinder</code> , <code>Rectangle</code>
└─ <code>com.iit.utils</code>	→ <code>MathHelper</code> , <code>StringUtils</code>
└─ <code>com.iit.database</code>	→ <code>Connection</code> , <code>Query</code> , <code>DatabaseManager</code>
└─ <code>com.iit.ui</code>	→ <code>Button</code> , <code>Window</code> , <code>Dialog</code>
└─ <code>com.iit.app</code>	→ <code>Main</code> , <code>Application</code>

Rules:

- All lowercase
 - Use dots (.) to separate levels
 - Start with reverse domain to ensure uniqueness
 - Be descriptive and hierarchical
-

5.3 File System Structure

Package declaration MUST match folder structure!

```
project/
├── src/
│   ├── com/
│   │   └── iit/
│   │       ├── shapes/
│   │       │   ├── Circle.java      (package com.iit.shapes;)
│   │       │   ├── Cylinder.java    (package com.iit.shapes;)
│   │       │   └── Rectangle.java    (package com.iit.shapes;)
│   │       ├── utils/
│   │       │   └── MathHelper.java   (package com.iit.utils;)
│   │       └── app/
│   │           └── Main.java          (package com.iit.app;)
└──
```

5.4 Creating and Using Packages

Circle.java (in com.iit.shapes package)

```
package com.iit.shapes; // ⚠ MUST be first statement (before imports)

/**
 * Circle class in com.iit.shapes package
 */
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getArea() {
        return Math.PI * radius * radius;
    }

    @Override
    public String toString() {
        return String.format("Circle[r=%.2f]", radius);
    }
}
```

Cylinder.java (in same package)

```
package com.iit.shapes; // Same package as Circle

public class Cylinder {
    private double radius;
    private double height;

    public Cylinder(double radius, double height) {
        this.radius = radius;
        this.height = height;
    }

    public double getVolume() {
        return Math.PI * radius * radius * height;
    }
}
```

5.5 Import Statements

Main.java (in different package)

```
package com.iit.app; // Different package

// =====
// IMPORT TYPES
// =====

// 1. SINGLE CLASS IMPORT (preferred - most specific)
import com.iit.shapes.Circle;
import com.iit.shapes.Cylinder;

// 2. WILDCARD IMPORT (imports all classes in package)
// import com.iit.shapes.*;

// 3. STATIC IMPORT (import static members)
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

public class Main {
    public static void main(String[] args) {
        // Can now use Circle without package prefix
        Circle c = new Circle(5);
        Cylinder cy = new Cylinder(3, 10);

        System.out.println(c.getArea());
        System.out.println(cy.getVolume());

        // Can use PI directly (because of static import)
        double area = PI * 5 * 5; // No Math. prefix needed
        double hypotenuse = sqrt(9 + 16); // No Math. prefix needed
    }
}
```

5.6 Import Styles Compared

```
// =====
```

```
// OPTION 1: Specific Imports (BEST PRACTICE)
// =====
import com.iit.shapes.Circle;
import com.iit.shapes.Cylinder;

public class Demo {
    Circle c = new Circle(5);
    Cylinder cy = new Cylinder(3, 10);
}

// =====
// OPTION 2: Wildcard Import (OK for small projects)
// =====
import com.iit.shapes.*;

public class Demo {
    Circle c = new Circle(5);
    Cylinder cy = new Cylinder(3, 10);
}

// =====
// OPTION 3: Fully Qualified Names (No import needed)
// =====
public class Demo {
    com.iit.shapes.Circle c = new com.iit.shapes.Circle(5);
    com.iit.shapes.Cylinder cy = new com.iit.shapes.Cylinder(3, 10);
}
// ^ Verbose but useful when two packages have same class name

// =====
// EXAMPLE: Resolving Name Conflicts
// =====
import java.util.Date; // One Date class

public class Demo {
    java.util.Date utilDate = new java.util.Date();
    java.sql.Date sqlDate = new java.sql.Date(0);
    // ^ Must use fully qualified name for sql.Date to avoid conflict
}
```

5.7 Built-in Packages

Java automatically imports `java.lang.*` - you never need to import:

- `String`
- `Math`
- `System`
- `Object`
- `Integer`, `Double`, `Boolean`, **etc.**
- `Exception`, `RuntimeException`
- **And many more...**

// No imports needed for these:

```
public class Demo {
    public static void main(String[] args) {
        String s = "hello";           // java.lang.String
        System.out.println(s);        // java.lang.System
    }
}
```



```

        int max = Math.max(5, 10);    // java.lang.Math
        Integer obj = 42;             // java.lang.Integer
    }
}

```

Common Java Packages:

java.lang.*	→ Core classes (String, Math, System) - auto-imported
java.util.*	→ Collections, Date, Scanner, Random
java.io.*	→ Input/Output (File, InputStream, OutputStream)
java.nio.*	→ New I/O (Path, Files)
java.net.*	→ Networking (URL, Socket)
java.sql.*	→ Database connectivity
javax.swing.*	→ GUI components
java.time.*	→ Date/Time API (Java 8+)

5.8 Static Imports

Static imports let you import static members (methods, constants) directly.

```

// =====
// WITHOUT STATIC IMPORT
// =====
public class Demo {
    public static void main(String[] args) {
        double area = Math.PI * 5 * 5;
        double root = Math.sqrt(16);
        int max = Math.max(10, 20);
    }
}

// =====
// WITH STATIC IMPORT
// =====
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;
import static java.lang.Math.max;

public class Demo {
    public static void main(String[] args) {
        double area = PI * 5 * 5;           // No Math. prefix
        double root = sqrt(16);
        int maximum = max(10, 20);
    }
}

// =====
// WILDCARD STATIC IMPORT (imports ALL static members)
// =====
import static java.lang.Math.*;

public class Demo {
    public static void main(String[] args) {
        double area = PI * pow(5, 2);      // Both PI and pow imported
        double root = sqrt(16);
    }
}

```

When to use static imports:

- ✓ Mathematical constants and functions (Math.PI, Math.sqrt)
 - ✓ Frequently used enums
 - ✓ Test assertions (JUnit)
 - ⚠ Use sparingly - can reduce code readability
-

5.9 Package Access Control

Classes and members have different visibility based on access modifiers:

```
package com.iit.shapes;

public class Circle {
    public double radius;           // Accessible EVERYWHERE
    protected double diameter;     // Package + subclasses
    double circumference;          // Package-private (no modifier)
    private double area;           // Only within Circle class
}
```

Visibility Table:

Modifier	Class	Package	Subclass	World
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
(default)	✓	✓	✗	✗
private	✓	✗	✗	✗

Example:

```
// In com.iit.shapes package:
public class Circle {
    public void publicMethod() { } // Accessible in com.iit.app ✓
    protected void protectedMethod() { } // NOT accessible in com.iit.app ✗
    void packageMethod() { } // NOT accessible in com.iit.app ✗
    private void privateMethod() { } // NOT accessible in com.iit.app ✗
}

// In com.iit.app package:
import com.iit.shapes.Circle;

public class Main {
    public static void main(String[] args) {
        Circle c = new Circle();
        c.publicMethod(); // ✓ OK
        // c.protectedMethod(); // ✗ ERROR
        // c.packageMethod(); // ✗ ERROR
        // c.privateMethod(); // ✗ ERROR
    }
}
```

}

5.10 Best Practices

✓ DO:

1. Always use package declarations
2. Use reverse domain naming
3. Prefer specific imports over wildcards in large projects
4. Organize classes logically (shapes, utils, database)
5. Make utility classes in their own package
6. Use meaningful package names

✗ DON'T:

1. Put everything in the default package (no package statement)
2. Mix unrelated classes in same package
3. Use wildcards when importing from many packages (unclear dependencies)
4. Create overly deep package hierarchies
5. Start package names with capital letters

💡 Key Takeaways: Part 5

Concept	Key Points
Package	Organizes classes into namespaces; prevents conflicts
Naming	Reverse domain: <code>com.company.project.module</code>
Structure	Package declaration must match folder structure
Import	Specific imports (preferred) or wildcard (*)
Static Import	Import static members for cleaner code
java.lang	Automatically imported (String, Math, System)

Summary & Quick Reference

Complete OOP Concepts Map

OOP FUNDAMENTALS (Circle)
• Class = Blueprint • Object = Instance of class • Constructor = Initializes state • this = Reference to current object • Assignment = Copies reference, not object (aliasing!) • Overloading = Same name, different parameters

STATIC vs INSTANCE (Cylinder)
• Static = Class-level (shared), access via ClassName • Instance = Object-level (per object), access via object • Static methods cannot access instance members • Use static for: constants, utilities, factories, counters
VARIABLES
• Instance: per object, default value, heap • Static: shared, default value, method area • Local: per method, NO default, must initialize, stack
STRING
• Immutable - methods return NEW strings • Use .equals() for comparison, NEVER == • StringBuilder for loops (mutable, efficient)
PREDEFINED CLASSES
• Math: static utilities (PI, sqrt, random, etc.) • Wrappers: Convert primitives ↔ objects, parsing
PACKAGES
• Organization: com.company.project • Import: import package.Class; • java.lang.* automatically imported

Common Mistakes to Avoid

✗ Mistake #1: String Immutability

```
String s = "hello";
s.toUpperCase(); // ✗ Does nothing - result not stored!
System.out.println(s); // Still "hello"
```

```
// ✓ CORRECT:
s = s.toUpperCase();
System.out.println(s); // "HELLO"
```

✗ Mistake #2: String Comparison with ==

```
String s1 = "hello";
String s2 = new String("hello");
if (s1 == s2) { } // ✗ Compares references, not content!
```

```
// ✓ CORRECT:
if (s1.equals(s2)) { }
```

✗ Mistake #3: Static Accessing Instance

```

public class Circle {
    private double radius;

    public static void printRadius() {
        System.out.println(radius); // ✗ ERROR: non-static variable
    }
}

// ✔ CORRECT: Make it instance method OR pass object as parameter
public void printRadius() {
    System.out.println(radius); // ✔ OK - instance method
}

```

✗ Mistake #4: Integer Division

```

public void scale(int percent) {
    this.radius *= (percent / 100); // ✗ 150/100 = 1 (integer division!)
}

// ✔ CORRECT:
this.radius *= (percent / 100.0); // 150/100.0 = 1.5

```

✗ Mistake #5: Local Variable Not Initialized

```

public void method() {
    int x;
    System.out.println(x); // ✗ ERROR: variable might not have been
    initialized
}

// ✔ CORRECT:
int x = 0; // Or assign before use
System.out.println(x);

```

✗ Mistake #6: Thinking Assignment Copies Objects

```

Circle c1 = new Circle(5);
Circle c2 = c1; // ✗ Both point to SAME object!
c2.setRadius(10);
System.out.println(c1.getRadius()); // Prints 10 (not 5!)

// ✔ To create a copy:
Circle c2 = new Circle(c1.getRadius()); // New object with same radius

```

✗ Mistake #7: Using += with StringBuilder

```

String result = "";
for (int i = 0; i < 1000; i++) {
    result += i; // ✗ Very slow - creates many String objects
}

// ✔ CORRECT:
StringBuilder sb = new StringBuilder();
for (int i = 0; i < 1000; i++) {
    sb.append(i);
}

```

```
}  
String result = sb.toString();
```

Practice Exercises

Exercise 1: Box Class

Create a `Box` class with the following requirements:

- Instance variables: `width`, `height`, `depth` (all double)
- Static variable: `count` (tracks total boxes created)
- No-arg constructor: creates 1×1×1 box
- Parameterized constructor: accepts `w`, `h`, `d`
- Method: `getVolume()` returns `width × height × depth`
- Static method: `getCount()` returns total boxes created
- Overloaded methods: `scale(double factor)` and `scale(int percent)`
- `toString()` method

Starter Code:

```
package com.iit.shapes;  
  
public class Box {  
    // Your code here  
}
```

Test Code:

```
Box b1 = new Box();  
Box b2 = new Box(2, 3, 4);  
System.out.println(b2.getVolume()); // Should print 24.0  
System.out.println(Box.getCount()); // Should print 2  
b2.scale(2.0);  
System.out.println(b2.getVolume()); // Should print 192.0
```

Exercise 2: String Manipulation

Write a method that:

1. Takes a `String` parameter
2. Removes all whitespace
3. Converts to uppercase
4. Returns the result

```
public static String process(String input) {  
    // Your code here  
}
```

```
// Test:
```

```
System.out.println(process(" Hello World ")); // "HELLOWORLD"
```

Exercise 3: StringBuilder Performance

Write two methods:

1. `concatWithString()` - uses `String +=` in a loop (1000 iterations)
2. `concatWithBuilder()` - uses `StringBuilder`

Measure and compare their execution times.

Exercise 4: Static Counter

Create a `Student` class that:

- Tracks total students created (static counter)
- Assigns unique ID to each student (using counter)
- Has instance variables: `name`, `id`
- Has static method: `getTotalStudents()`

```
Student s1 = new Student("Alice"); // ID = 1
Student s2 = new Student("Bob");   // ID = 2
System.out.println(Student.getTotalStudents()); // 2
```

Exercise 5: Package Organization

Organize these classes into appropriate packages:

- `Circle`, `Rectangle`, `Triangle`
- `MathHelper`, `StringHelper`
- `Main`

Create the folder structure and add proper package declarations and imports.

Additional Resources

Recommended Reading

1. **Oracle Java Tutorials:** <https://docs.oracle.com/javase/tutorial/>
2. **Effective Java** by Joshua Bloch (Chapter 2: Creating Objects)
3. **Java: The Complete Reference** by Herbert Schildt

Online Practice

1. **CodingBat** (Java): <https://codingbat.com/java>
2. **HackerRank** - Java OOP track
3. **LeetCode** - Java basics

Next Topics

- Inheritance & Polymorphism
 - Abstract Classes & Interfaces
 - Exception Handling
 - Collections Framework
 - File I/O
-

Study Checklist

Before the exam, make sure you can:

- ☐ Write a complete class with constructors, getters, setters
 - ☐ Explain the difference between class and object
 - ☐ Demonstrate object assignment and aliasing
 - ☐ Implement method overloading correctly
 - ☐ Distinguish instance, static, and local variables
 - ☐ Write and use static methods properly
 - ☐ Explain static context restrictions
 - ☐ Use String methods and understand immutability
 - ☐ Use StringBuilder for efficient concatenation
 - ☐ Apply Math class methods
 - ☐ Parse strings to primitives using wrapper classes
 - ☐ Organize code using packages
 - ☐ Write correct import statements
-

End of Lecture Notes

Good luck with your studies! Remember: practice is key to mastering OOP concepts. ☐